ProEd Computer Science

Report on Q-Learning, Neural Networks & AI/ML

report title: **I spent 1 day of my life on this stupid crying baby**

This is agent and he's stupid.

(For now)

We have to train him first.

The agent learns to navigate the environment using Q-Learning. The agent receives rewards for reaching the present and penalties for falling into pools. The Q-values are updated based on the agent's actions and the rewards received, allowing it to learn the optimal path over time.

The agent learns to navigate the environment using Q-Learning. Essentially, we're teaching a baby how to walk across the lake. And the first step is to throw him out there, abandon him and hope for the best. 🤞

Oh look, he's doing... horribly.

The agent receives rewards for reaching the present and penalties for falling into pools. The Q-values are updated based on the agent's actions and the rewards received, allowing it to learn the optimal path over time.

Slowly, (but surely), the agent gains... *awareness*.

Instead of being a robot that runs `random.randint(1, 4)` every move, it is now: a robot that runs `random.randint(1, 4)` *almost* every move, due to the decreasing *epsilon*. Except a few, where it uses what it knows about its future experiences (which it has stored within its Q-table), and chooses the best option that it has done before.

Now, it's actually learning. Our agent uses what it knows (the current state, the action taken and the reward from that action) to create a large (not actually *that* large) database of which moves led to the best results.

And from here, practice makes perfect, and we force our agent to do the same maze over and over again, breaking some labor laws, workers' rights laws, some people's morals, but, saving the most important part for last, we have our trained agent.

Hyperparameters
There are certain values, or hyperparameters, that can be tweaked to alter the learning of the agent.

Learning Rate (α) defines how quickly the agent accepts new information, dictating how *trusting* it is of new information it receives.

Discount Rate (γ) defines how much the agent values long-term reward over short-term rewards. For us, this is set pretty high as we want the agent to prioritize the long-term goal of reaching the present at the end.

Epsilon Curve defines how the ε-value changes over time. A ε-value of 1 means the agent moves completely randomly, and

0 means it only refers to its Q-table. Personally, I set the ε-value to be: $\varepsilon_n = 0.99^n$

The Reward Function defines how the agent is rewarded for every move. You can train the agent to have different goals by altering this function.
This is set to: +2 for reaching the end, since we want to reward it highly for winning the maze; -2 for walking into a pool, as dying is quite unhealthy (and doesn't contribute to our goal); and a smaller bonus for moving closer to the goal, paired with a 'time penalty' which constantly reduces its score. (I found that this was necessary as otherwise, the agent would simply move in between two spots, increasing its score infinitely).

Q-Tables, States and Actions
Q-Tables represent all possible states and actions, stored within a table. Since we are dealing with one-dimensional states and actions, we can have a 2-dimensional table matching all possible states with all possible actions.

This is where our agent stores its own 'memory', remembering all of its previous actions, how they ended up and is where the agent also chooses its next action.

I also did some research onto deep q-learning (but I couldn't get PyTorch working on my laptop)
Like seriously, why do different people and websites refer to conda-forge::py-box2d, py-box2d, gymnasium[box2d], gym[box2d], box2d, and "gymnasium[box2d]"??? But only one (or maybe a few) of them works???

So I guess I'll just yap about some stuff. **The rest of this is not necessary to read** (its very boring). its just my current understanding of how these things work.

Deep Q-Learning is a type of Q-Learning that uses Neural Networks instead of a Q-Table, which can approximate Q-values for different states and actions. The benefit of this is since this is a neural network, it can accept continuous states as input, and output any value between 0 and 1. Additionally, this allows us to work with high-dimensional data, as we can essentially use as many neurons as we want for our input, and as many as we want for our output.

Goal of the Neural Network: *a function, Q^* that allows us to determine how good an action is in a certain situation.*

$$Q^* : State \times Action \rightarrow \mathbb{R}$$

Neural Networks are good *approximations* of functions. This means that although we don't have access to the exact function Q^*, we can use a neural network that approximates this function.

So, how can we create the network that can approximate this function in the first place?

The **Bellman Equation**, states that the **value,** or **reward** of an action (taken at a certain state) can be expressed as some function of the **immediate** reward, and the **expected** reward of the **next state we end up in** (this value is dependent on gamma, or the discount rate).

All Q-functions, obey this law. So, this allows us to find the **difference** between ***our*** **version of the Q-function,** and **the actual Q-function** we are trying to find.

The actual Q-function? How do we know that? Isn't this what we're trying to find?
Actually, we *don't* know the actual Q-function. That's why we need to approximate it. However, the Bellman Equation allows us to create a "target point", that *our* Q-function is constantly working towards.

For any state-action pair (s, a); a target value can be create through:
The immediate reward, R, we get from taking a in s, and;
The maximum Q-value for the preceding state, s'
Thus,
$\text{target} = r + \gamma * \max(Q(s', a'))$

Finally, the comparison between them can be made between our current network's prediction, $Q(s, a)$ and the *target*, calculated above.

An equation calculating the difference between the two is called the **Huber Loss**.

But first, why not just use the 'normal' loss functions? Firstly, MSE is limited by how large errors have disproportionate impacts. While it functions for smaller errors, extremely large values, or anomalous results have too large an impact. Similarly, MAE is not sensitive enough to small errors, which leads to slower training.

The Huber loss fixes both of these limitations. It is similar to MSE, if the error is small, while being more similar to MAE for larger ones. This means the equation for the Huber loss consists of one equation if $\delta \leq 1$, and another otherwise.

$$\mathcal{L} = \frac{1}{|B|} \sum_{(s,a,s',r) \in B} \mathcal{L}(\delta)$$

$$\text{where} \quad \mathcal{L}(\delta) = \begin{cases} \frac{1}{2}\delta^2 & \text{for } |\delta| \leq 1, \\ |\delta| - \frac{1}{2} & \text{otherwise.} \end{cases}$$

Now, once the Huber loss has been calculated, we can calculate a gradient descent: essentially, it shows us how we need to adjust each of our weights, gradually improving the network over time.

Repeat this (many, many) times, and you have a trained network.